LIME and SHAP: Explaining Machine-Learning Predictions

Seminar S04 Report – CS420

Nguyễn Trọng Hoàng 24125056
Phan Minh Khôi 24125061
Nguyễn Thuận Việt 24125051

Abstract

Machine-learning models are increasingly used in high-stakes fields such as medicine, yet a model that is usually right does not automatically reveal why it made a particular decision. This report asks whether a highly accurate prediction necessarily comes with a reliable explanation, focusing on two widely used tools, LIME (Local Interpretable Model-agnostic Explanations) and SHAP (SHapley Additive exPlanations), which estimate how much each input feature contributed to a single prediction. We apply both tools to two models trained to detect breast cancer from tumour measurements and examine an easy, a borderline and a misclassified case, asking whether the tools agree, whether their results stay the same when re-run, and whether the features they rank highest truly drive the prediction. Although the models are 98.2% and 95.6% accurate, the tools agree well only on the easy case: on the misclassified case they share just two of their five most important features, and their rankings shift noticeably between runs. Both tools find influential features far better than random choice, but neither is clearly more reliable than the other. We therefore recommend repeating explanations and checking their stability before trusting them, especially for uncertain or incorrect predictions.

1 Introduction

Many modern machine-learning models are "black boxes": we see the input and the output but not which features drive a decision. In healthcare, finance or autonomous driving, a prediction cannot be trusted just because it is usually right, so the need to explain individual decisions keeps growing [1][2]. LIME [1] and SHAP [3] are two widely used post-hoc methods for this purpose.

Guiding question. Does a highly accurate prediction necessarily come with a reliable explanation? To answer it, we look at three criteria: whether an explanation is useful (it shows which features push the prediction up or down), stable (it does not change with the seed or the background data) and faithful to the model (removing features called important should change the prediction accordingly).

Scope. We use the Breast Cancer Wisconsin dataset [4] (569 instances, 30 features, two classes), two models (Logistic Regression and Random Forest) and three instances chosen by a fixed rule. The two source papers are Ribeiro et al. [1] for LIME and Lundberg & Lee [3] for SHAP; the survey by Mersha et al. [2] is used to place both methods in the XAI taxonomy and to choose evaluation criteria.

Contributions. We (i) synthesise and compare the two required papers [1][3] and one survey [2]; (ii) work through SHAP on a linear model and LIME on a small non-linear model by hand; (iii) compare the two methods on top-k overlap, runtime and stability across seeds and background sets; and (iv) run a deletion test and propose a procedure for evaluating faithfulness. What belongs to the papers carries a citation; the remaining examples and numbers are our own.

2 Background and Paper Review

2.1 Core concepts

Mersha et al. [2] point out that the terminology of XAI is not yet settled. In this report, *interpretability* means how well a person can understand how a model works internally, and *explainability* means the ability to give reasons for *one specific decision*. Some authors treat one term as a subset of the other, so we state our definitions explicitly. Table 1 lists the concept pairs used in the rest of the report and shows where LIME and SHAP fall.

Table 1: Core concept pairs and where LIME and SHAP fall.

Concept pair	Distinction	LIME and SHAP
Explainability – interpretability	Explaining one prediction vs. understanding how the model works overall	Provide explainability; do not make the model interpretable
Local – global	Explaining one instance vs. describing the whole model	Both are local; LIME adds SP-LIME to pick representative instances for a global view [1]
Model-specific – model-agnostic	Depends on model structure vs. needs only the prediction function	LIME and KernelSHAP are model-agnostic; Linear SHAP and Deep SHAP are model-specific [3]
Intrinsic (ante-hoc) – post-hoc	Model is understandable by design vs. explained after training	Both are post-hoc and perturbation-based [2]

2.2 LIME

LIME, proposed by Ribeiro et al. [1], explains a single prediction by fitting a simple model that imitates the black box near the instance of interest. It samples perturbed points around an input x, queries the black-box model at each point, and fits an interpretable surrogate in which points closer to x count more. Formally, the explanation is

$$\xi(x) = \arg\min_{g \in G} \; L(f, g, \pi_x) + \Omega(g), \tag{1}$$

$$L(f, g, \pi_x) = \sum_{z, z'} \pi_x(z)\big(f(z) - g(z')\big)^2, \qquad \pi_x(z) = \exp\big(-D(x, z)^2/\sigma^2\big). \tag{2}$$

Here f is the black-box model; in our experiments $f(x) \in [0, 1]$ is the predicted probability of one class. $x \in \mathbb{R}^d$ is the instance being explained ($d = 30$ in our data), $z \in \mathbb{R}^d$ is a perturbed sample in the original feature space, and $z' \in \{0, 1\}^N$ is its interpretable representation over N interpretable components, where $z'_i = 1$ if the i-th component is present in z and $z'_i = 0$ otherwise. G is a family of interpretable models (here linear models $g(z') = w_g \cdot z'$), $D \geq 0$ is a distance function (Euclidean in our worked example) and $\sigma > 0$ is the kernel width, so $\pi_x(z) \in (0, 1]$ and $L \geq 0$ is measured in squared probability units. $\xi(x)$ is the chosen surrogate, whose weights w_g are the explanation.

Two terms in Eq. (1) shape the explanation:

- The proximity kernel $\pi_x(z)$ defines what "local" means: the closer a perturbed sample is to x, the more weight it receives. The kernel width σ sets the size of this neighbourhood. If σ is too small, only a few samples carry meaningful weight and the explanation becomes noisy; if it is too large, the surrogate is no longer local.
- The complexity penalty $\Omega(g)$ keeps the explanation short. For linear surrogates Ribeiro et al. set $\Omega(g) = \infty \cdot \mathbb{1}\big[\|w_g\|_0 > K\big]$, which forbids more than K non-zero weights, and use K-LASSO to select the K features to keep. This trades some local fidelity for readability.

2.3 SHAP

SHAP, introduced by Lundberg and Lee [3], starts from a class of methods the authors call *additive feature attribution* methods. In this class, the explanation model is a linear function of binary variables:

$$g(z') = \varphi_0 + \sum_{j=1}^{M} \varphi_j z'_j, \qquad z' \in \{0,1\}^M, \tag{3}$$

where φ_j is the attribution of feature j and M is the number of features ($M = 30$ in our data). The attributions have the same unit as f: in our experiments $\varphi_0 \in [0,1]$ and each $\varphi_j \in [-1,1]$ is a change in probability. The authors prove (Theorem 1) that only one choice of attributions in this class satisfies three properties:

- *local accuracy*: the explanation reproduces the prediction, $g(x') = f(x)$;
- *missingness*: a feature that is absent receives no attribution;
- *consistency*: if the model changes so that a feature's marginal contribution does not decrease, its attribution does not decrease either.

This unique solution is the Shapley value:

$$\varphi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(M - |S| - 1)!}{M!} \Big[f_{S\cup\{j\}}(x_{S\cup\{j\}}) - f_S(x_S) \Big]. \tag{4}$$

Here F is the set of all M features, S is a subset that does not contain feature j, and f_S is the prediction when only the features in S are known; the remaining features are averaged out over a set of background data. Three quantities matter when reading a SHAP explanation:

- **Baseline.** $\varphi_0 = \mathbb{E}[f(X)]$ is the average prediction over the background data, so it depends on how that data is chosen.
- **Contribution.** φ_j is the share of the gap between $f(x)$ and the baseline that feature j accounts for; together, $\varphi_0 + \sum_j \varphi_j = f(x)$ exactly.
- **Sign.** A positive φ_j pushes the prediction above the baseline, and a negative one pushes it below.

KernelSHAP estimates Shapley values with a weighted linear regression. Lundberg and Lee show that this is LIME with a specific choice of kernel, loss and regulariser (Theorem 2), and they note that LIME's heuristic choices of these components can violate local accuracy or consistency. The estimation usually assumes that features are independent, and some variants also assume that the model is linear. For a linear model with independent features, the Shapley values have the closed form $\varphi_j = w_j(x_j - \mathbb{E}[X_j])$, which is the basis of the worked example in Section 3.2.

2.4 The survey by Mersha et al.

Mersha et al. [2] classify XAI techniques along four axes: scope (local or global), stage (ante-hoc or post-hoc), applicability (model-specific or model-agnostic) and design methodology (perturbation-based or gradient-based). To evaluate explanations, they separate human-centred approaches from computer-centred ones and list five computer-centred criteria: fidelity, consistency, robustness, efficiency and sufficiency. They also stress that no single metric suits every system. We use these criteria to organise the evaluation in Section 4.

2.5 Comparing the three sources

Table 2 compares the goal, main contribution and limitations of the three sources.

Table 2: Comparison of the three main sources.

Source	Goal	Main contribution	Limitations / notes
Ribeiro et al. [1]	Explain the predictions of any classifier so that people can decide whether to trust it	LIME (a sparse, locally weighted surrogate) and SP-LIME for picking representative instances	Kernel and σ are chosen heuristically; no Shapley-style guarantees
Lundberg & Lee [3]	Unify existing feature-attribution methods	The additive attribution class, its unique solution (Shapley values) and KernelSHAP	Estimation assumes feature independence; results depend on the background data
Mersha et al. [2]	Survey XAI: taxonomy, applications and evaluation	A four-axis taxonomy and five evaluation criteria	Covers LIME and SHAP only briefly; we use its criteria in spirit, not its exact formulas (Section 4.7)

3 Method and Worked Examples

We describe the setup, then work through one small example per method so that the computations behind Section 4 can be followed.

3.1 Experimental setup

We use the Breast Cancer Wisconsin data [4] from scikit-learn (class 0 = malignant), an 80/20 stratified split (455 train, 114 test), a standardised Logistic Regression (LR) and a 200-tree Random Forest (RF), with test accuracy 98.2% and 95.6%. Both methods explain $P(\text{malignant})$: SHAP with KernelSHAP (100 background instances, 2000 coalitions), LIME with the library defaults (5000 samples, kernel width ≈ 4.11). Every choice was fixed before looking at the results and all seeds are fixed; Appendix A gives the full setup and the reason for each choice.

Features. The 30 features are computed from a digitised image of a fine-needle aspirate: ten cell-nucleus measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension), each given as the mean, the standard error ("error") and the mean of the three largest values ("worst"). All values are non-negative and no physical unit is given in the data: the size features are on the image scale (area in squared units of radius) and the others are dimensionless ratios or grey-level statistics. Their ranges differ widely, for example mean radius 6.98–28.11, worst area 185–4254 and mean smoothness 0.053–0.163, which is why LR uses standardised inputs.

Instances. A fixed rule applied to the RF picks three test instances (Table 3): the correct prediction with the highest confidence, the prediction with the smallest $|P - 0.5|$ and the first misclassified one. This prevents us from choosing favourable examples.

Table 3: The three chosen instances. $P > 0.5$ means the model predicts malignant.

Instance	Test index	True label	P(malignant), RF / LR
Confidently correct	0	malignant	1.000 / 1.000
Borderline	16	benign	0.495 / 0.634 (LR is wrong)
Misclassified	3	benign	0.745 (RF is wrong) / 0.466

3.2 Worked example 1: SHAP on a linear model

Before applying SHAP to real data, we compute it by hand on a small linear model to show how the baseline and the feature contributions are obtained and to check that they add up to the prediction (local accuracy, Section 2.3). Assume the model $f(x) = b + \sum_j w_j x_j$, independent features, and the background is the mean $\mu_j = \mathbb{E}[X_j]$. Then $\varphi_0 = f(\mu)$ and $\varphi_j = w_j(x_j - \mu_j)$. Take $b = 1$, $w = (2, -1, 0.5)$, $\mu = (1, 2, 4)$ and the instance $x = (3, 1, 6)$.

- Baseline: $\varphi_0 = f(\mu) = 1 + 2 \cdot 1 - 1 \cdot 2 + 0.5 \cdot 4 = 3$.
- Prediction: $f(x) = 1 + 2 \cdot 3 - 1 \cdot 1 + 0.5 \cdot 6 = 9$.
- Contributions: $\varphi_1 = 2(3 - 1) = 4$; $\varphi_2 = (-1)(1 - 2) = 1$; $\varphi_3 = 0.5(6 - 4) = 1$.
- Check: $\varphi_0 + \varphi_1 + \varphi_2 + \varphi_3 = 3 + 4 + 1 + 1 = 9 = f(x)$.

Enumerating every subset with the original Shapley formula (with $v(S) = f$ where features in S take their value from x and the rest take the mean) gives the same result $(4, 1, 1)$. Feature 2 has $w_2 < 0$ but $x_2 < \mu_2$, so its contribution is still positive: the sign of φ_j depends on both the coefficient and where the instance lies relative to the background.

3.3 Worked example 2: LIME step by step

We now run LIME by hand on a small non-linear model to show its three steps (perturbing the input, weighting the samples by distance, and fitting a weighted linear surrogate) and to compare the resulting coefficients with the model's true local slopes.

Model and instance. We use the model

$$f(x) = \text{sigmoid}(x_1^2 - x_2), \qquad \text{sigmoid}(t) = \frac{1}{1 + e^{-t}}. \tag{5}$$

The sigmoid function maps any real number t to a value between 0 and 1, so $f(x)$ can be read as a probability, as in logistic regression. The model is non-linear in its inputs, both because of the sigmoid and because of the x_1^2 term, so a linear surrogate can only approximate it locally. The instance to explain is $x_0 = (1, 2)$, for which $f(x_0) = \text{sigmoid}(1 - 2) = \text{sigmoid}(-1) = 0.2689$.

To keep the example traceable, we make three simplifications: we use 6 fixed perturbed points instead of random sampling, set the kernel width to $\sigma = 1$, and drop the complexity penalty Ω. We also use the original features (shifted by x_0) as the interpretable representation instead of the binary vector z' of Section 2.2, so the coefficients can be compared directly with the slopes of f.

Step 1: perturb and query. For each perturbed point z, compute the model output $f(z)$. For example, for $z = (1.3, 2.2)$ we get $f(z) = \text{sigmoid}(z_1^2 - z_2) = \text{sigmoid}(-0.51) = 0.3752$.

Step 2: weight by distance. Compute the distance $D = \|z - x_0\|$ and the weight $\pi = \exp(-D^2/\sigma^2)$. For $z = (1.3, 2.2)$, $D = \sqrt{(1.3 - 1)^2 + (2.2 - 2)^2} = 0.361$ and $\pi = 0.878$. Points closer to x_0 receive weights closer to 1. Table 4 lists all six points.

Step 3: fit a weighted linear surrogate. Fit $g(z) = \beta_0 + g_1(z_1 - 1) + g_2(z_2 - 2)$ by weighted least squares:

$$\beta = (A^\top W A)^{-1} A^\top W y, \tag{6}$$

where each row of A is $[1,\ z_1 - 1,\ z_2 - 2]$, $W = \text{diag}(\pi)$ holds the weights and y holds the six values $f(z)$. The coefficients g_1 and g_2 are LIME's explanation: how much the prediction changes per unit change in each feature near x_0.

Table 4: The six fixed perturbed points of the LIME example.

z_1	z_2	$f(z)$	D	π
1.3	2.2	0.3752	0.361	0.878
0.6	2.1	0.1493	0.412	0.844
1.2	1.5	0.4850	0.539	0.748
0.9	1.7	0.2911	0.316	0.905
1.5	2.4	0.4626	0.640	0.664
0.7	2.5	0.1182	0.583	0.712

Result. With $\sigma = 1$, LIME gives $g_1 = 0.395$ and $g_2 = -0.149$. The true slopes of f at x_0 are

$$\frac{\partial f}{\partial x_1} = f(1-f) \cdot 2x_1 = 0.393, \qquad \frac{\partial f}{\partial x_2} = -f(1-f) = -0.197, \tag{7}$$

using the identity $\text{sigmoid}'(t) = \text{sigmoid}(t)\big(1 - \text{sigmoid}(t)\big)$. The estimate g_1 is almost exact. The estimate g_2 has the right sign but is off by about 0.05, because only 6 points are used and f curves within the neighbourhood. This shows on a small scale that a LIME explanation depends on the perturbation sample. Changing the kernel width to $\sigma = 0.3$ gives $(g_1, g_2) = (0.362, -0.132)$, and $\sigma = 3$ gives (0.397, −0.150). In this tiny example σ matters little; Section 4 measures its effect on the 30-dimensional data.

4 Results and Discussion

4.1 Feature-contribution charts

Figure 1 (Random Forest) and Figure 2 (Logistic Regression) show the contributions of the 8 largest features. Orange pushes $P(\text{malignant})$ up, blue pushes it down. For SHAP, $\varphi_0 + \sum_j \varphi_j$ equals $f(x)$ exactly in all six cases (for example Random Forest, instance 0: $0.326 + 0.674 = 1.000$), as local accuracy requires.

- **Confidently correct instance (RF):** LIME and SHAP both put the tumour-size features (worst area, worst perimeter, worst radius, worst concave points) first, all pushing towards malignant.
- **Borderline instance (RF, $P = 0.495$):** contributions are small and mixed in sign; LIME puts worst concavity, mean texture and worst texture first, while SHAP puts worst perimeter, mean concave points (negative) and worst concavity first.
- **Misclassified instance (RF, $P = 0.745$):** see Section 4.3.

4.2 LIME versus SHAP

Overlap@5 is the number of features shared by the LIME and SHAP top-5 sets (ranked by |contribution|), divided by 5; it ranges from 0 (none shared) to 1 (the same five, in any order). We report it for the main run and as the average over all pairs of LIME and SHAP runs. *Spearman's* $\rho \in [-1, 1]$ compares how the two methods rank all 30 features by |contribution| (−1 reversed, 0 unrelated, 1 identical), so unlike Overlap@5 it uses the order of every feature. We also record each method's top feature and runtime (Table 5).

Agreement falls as the instance gets harder. For both models, Overlap@5 drops from 0.8–1.0 on the confidently correct instance to 0.4 on the misclassified one. When a feature appears in both top-5 sets, the two methods always give it the same sign (sign agreement is 1.0 in all six cases). The disagreement is therefore about *which* features are selected, not about the direction of their effect.

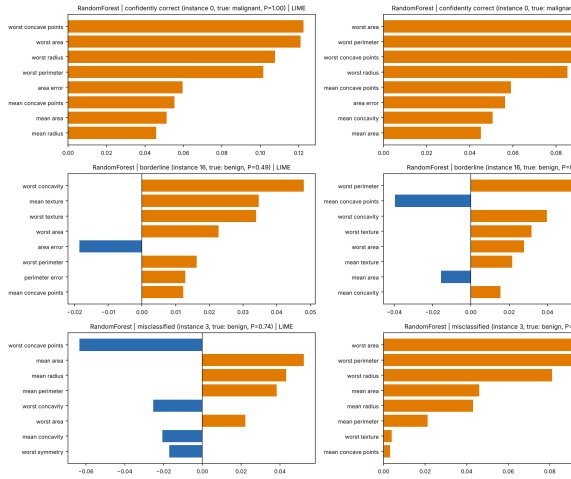


Figure 1: Feature contributions from LIME (left column) and SHAP (right column) for Random Forest; the three instances are, top to bottom, confidently correct, borderline and misclassified. Source: the team's experiment (code repository).

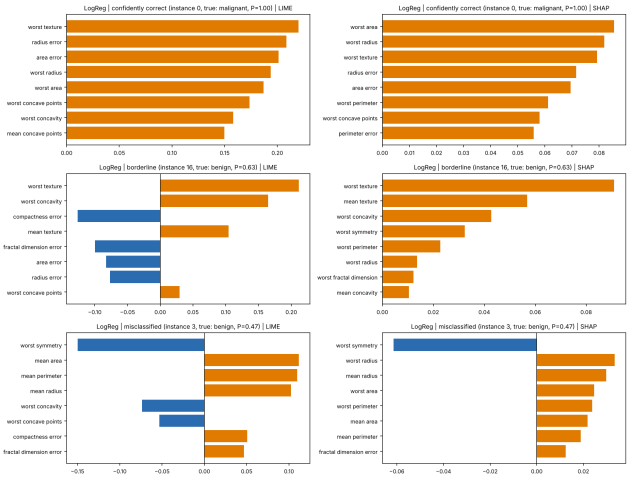


Figure 2: Same as Figure 1 for Logistic Regression. On instance 3 Logistic Regression predicts correctly ($P = 0.466$). Source: the team's experiment (code repository).

Table 5: LIME versus SHAP on the three instances. Overlap@5 is given as main run / average over all run pairs; runtimes are given as LIME / SHAP.

Model	Instance	Overlap@5	Spearman's ρ	Top-1 LIME	Top-1 SHAP	Runtime (s)
LogReg	Confidently correct	1.0 / 0.80	0.74	worst texture	worst area	0.035 / 0.043
LogReg	Borderline	0.6 / 0.67	0.24	worst texture	worst texture	0.037 / 0.035
LogReg	Misclassified	0.4 / 0.49	0.26	worst symmetry	worst symmetry	0.040 / 0.034
Random Forest	Confidently correct	0.8 / 0.82	0.83	worst concave points	worst area	0.066 / 0.441
Random Forest	Borderline	0.6 / 0.71	0.69	worst concavity	worst perimeter	0.067 / 0.484
Random Forest	Misclassified	0.4 / 0.44	0.46	worst concave points	worst area	0.069 / 0.390

Rankings differ most on the simplest model. Spearman's ρ is lowest for Logistic Regression on the two hard instances (0.24 and 0.26), so the full rankings differ substantially even for a linear model, and the top-5 sets overlap only in part (0.4–0.6). One possible, untested reason is that LIME discretises continuous features by default.

Runtime. On Logistic Regression the two methods take about the same time (about 0.04 s). On Random Forest, KernelSHAP is about 6–7 times slower than LIME (0.39–0.48 s vs. about 0.07 s), because it queries the model on 2000 coalitions $\times$ 100 background instances. All timings come from a single machine and varied by up to about 20% between our runs.

4.3 A case where the explanations disagree

We now look in detail at the misclassified instance (test index 3), where the Random Forest predicts $P(\text{malignant}) = 0.745$ although the true label is benign. Table 6 lists the top-5 features that each method gives for this prediction. A positive contribution pushes the prediction towards malignant, and a negative one pushes it towards benign.

The two methods share only two features (mean area and mean radius), so Overlap@5 is 0.4, and they tell different stories about the same prediction. SHAP attributes the prediction almost entirely to size features, all of which push towards malignant. LIME gives a mixed picture: its most important feature,

Table 6: Top-5 features of LIME and SHAP for the misclassified instance (Random Forest, test index 3). Features shared by both methods are in bold.

Method	Top-5 by |contribution|	Signs
LIME	worst concave points, **mean area**, **mean radius**, mean perimeter, worst concavity	worst concave points and worst concavity negative; the rest positive
SHAP	worst area, worst perimeter, worst radius, **mean area**, **mean radius**	All positive

worst concave points, pushes towards benign, while several size features push towards malignant.

No ground truth tells us which explanation is correct. One reading is that the methods answer different questions: LIME asks which value ranges matter near this instance, SHAP measures contributions relative to the average prediction over the background data. Either way, two widely used methods conflict on a wrong prediction, exactly where a user would rely on an explanation, so explanations must be checked for faithfulness (Section 4.7).

4.4 Stability

An explanation is stable if running the method again with different random choices gives a similar result. For LIME we vary the random seed (20 runs); for SHAP we vary the background set (5 runs). We use two measures. The *Jaccard similarity* of two top-5 sets A and B is $|A \cap B|/|A \cup B|$, and we report its mean over all pairs of runs; it equals 1 when every run selects the same five features. Note that Jaccard is stricter than Overlap@5: two sets sharing four of five features have Jaccard $4/6 \approx 0.67$ but Overlap@5 $= 0.8$. *Top-1 dominance* is the share of runs in which the top-ranked feature is the one most often ranked first; it equals 1 when every run agrees on the top feature. Table 7 shows the results.

Table 7: Stability of the top-5 features (LIME: 20 seeds; SHAP: 5 background sets).

Model	Instance	Jaccard LIME	Top-1 LIME	Jaccard SHAP	Top-1 SHAP
LogReg	Confidently correct	0.78	0.95	0.68	0.40
LogReg	Borderline	0.84	1.00	0.73	1.00
LogReg	Misclassified	0.85	1.00	0.53	1.00
Random Forest	Confidently correct	0.94	0.60	1.00	1.00
Random Forest	Borderline	0.72	0.80	0.73	1.00
Random Forest	Misclassified	0.74	1.00	1.00	1.00

Neither method is consistently more stable. The mean Jaccard ranges from 0.72 to 0.94 for LIME and from 0.53 to 1.00 for SHAP: SHAP is fully stable on two RF instances, yet also has the lowest value in the table (LR, misclassified instance).

The top feature can change even on easy instances. Top-1 dominance is low in two cells: 0.60 for LIME on the Random Forest and 0.40 for SHAP on Logistic Regression, both on the confidently correct instance. A possible explanation, which we did not test, is that when P is close to 1 many features contribute similar amounts, so small changes are enough to swap the top rank.

An unstable example. Figure 3 shows how often each feature enters the top-5 for the Random Forest on the borderline instance. Both methods almost always include worst concavity, worst texture and worst perimeter; they differ in ranks 4 and 5. LIME selects mean texture in 100% of runs and area error in 45%, whereas SHAP selects mean concave points in 100% of runs (LIME only in 10%) and mean texture in just 40%. LIME's top feature also changes between seeds: seed 0 ranks worst concavity first, while seeds 1–5 rank worst texture first.

Tuning does not remove the instability. Varying the SHAP background size and the LIME kernel

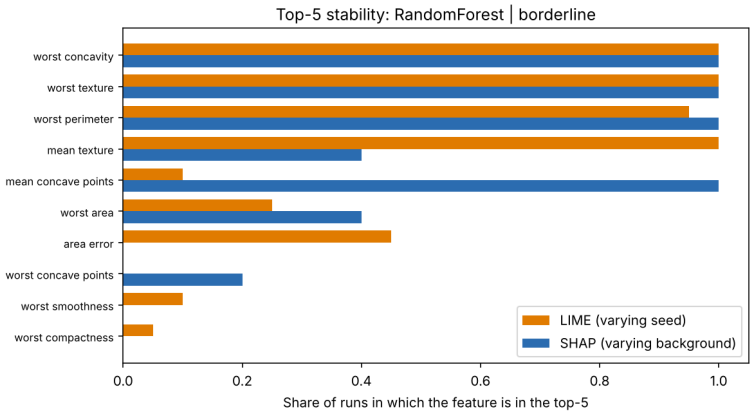


Figure 3: Share of runs in which each feature appears in the top-5, Random Forest, borderline instance (LIME: 20 seeds; SHAP: 5 background sets). Source: the team's experiment (code repository).

width on this instance (Appendix B) never lifts the Jaccard similarity above 0.73.

4.5 Checking faithfulness with a deletion test

An explanation is faithful if the features it marks as important really drive the model's prediction. We test this by deleting features and watching how the prediction changes. For each of 20 randomly chosen test instances, we take the features that the explanation says support the predicted class most strongly, remove them one at a time (up to 10) by replacing each with its training-set mean, and record the model's probability for the predicted class after each removal. If the explanation is faithful, this probability should fall quickly. As a baseline, we remove features in random order and average over 10 random permutations. Figure 4 shows the average curves.

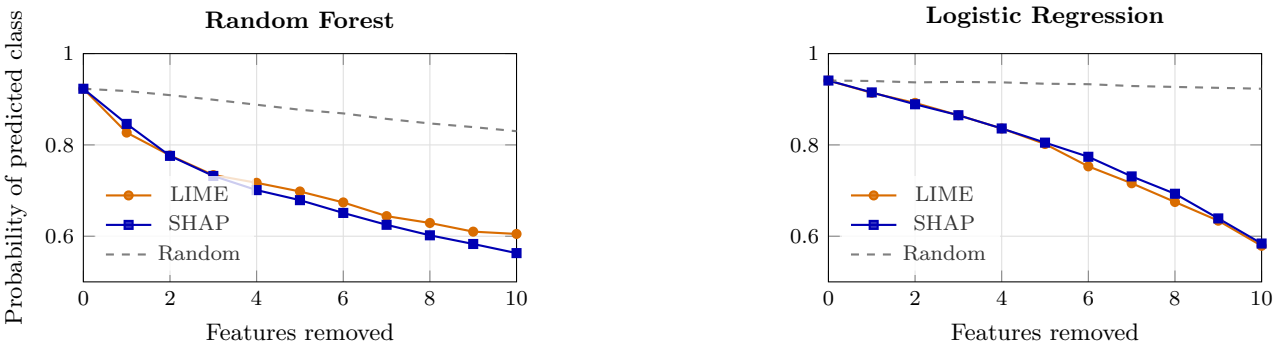


Figure 4: Deletion test: average probability of the predicted class as the top-ranked features are removed (20 test instances, one seed). A faster drop means a more faithful explanation. Source: the team's `results/deletion_curves.csv` (code repository).

Both LIME and SHAP lower the probability much faster than random removal. After 5 removals, Random Forest falls from 0.923 to 0.698 (LIME) and 0.679 (SHAP) but only to 0.877 at random; Logistic Regression falls from 0.941 to 0.802, 0.805 and 0.934. LIME and SHAP differ by at most 0.02 at 5 features and by 0.04 at 10 (RF: 0.605 vs. 0.563), which, with one seed, 20 instances and no confidence intervals, is not enough to rank them. Caveats: mean replacement can create out-of-distribution samples, and with strongly correlated features removing one does not remove its information.

4.6 Feature importance is not a causal effect

LIME and SHAP describe how the *model* uses its features, not how those features affect the *disease*: a large contribution means the model relies on a feature, not that it causes malignancy. Four reasons,

each visible in our experiment, keep the two apart.

- **Correlated features.** Worst radius, perimeter and area grow together and all appear near the top of the charts; how the credit is split among them depends on the model and the background data, not on which one is the real cause.
- **Unrealistic samples.** Both methods change each feature independently (for KernelSHAP, the independence assumption of Section 2.3), creating combinations that never occur in real tumours, such as a large area with a small radius.
- **Explanations also reflect the model's mistakes.** On the misclassified instance (Section 4.3) the size features explain why the model erred, not that the tumour is malignant.
- **Replacing a value is not a real intervention.** It is a computation on the model; knowing what would happen if tumour size actually changed requires causal assumptions that neither method tests.

Conclusions should therefore be phrased as "the model relies on X", not as "X causes Y".

4.7 A procedure for evaluating faithfulness (proposed)

Since LIME and SHAP can disagree and change between runs, we propose the five-step procedure below, organised (in spirit, not by its exact formulas) around the criteria of Mersha et al. [2]. Each step names its criterion and whether we carried it out.

1. **Sanity check with a known answer** (fidelity). For a linear model, the explanation must match $w_j(x_j - \mu_j)$. *Status:* done on the worked example (Section 3.2), not yet on the experiment's LR.
2. **Local fit** (fidelity). LIME: weighted R^2 of the surrogate around x; SHAP: $\varphi_0 + \sum_j \varphi_j = f(x)$. *Status:* SHAP checked (Section 4.1); LIME R^2 not computed.
3. **Deletion and insertion vs. random order** (fidelity, sufficiency). *Status:* deletion done (Section 4.5); insertion not done.
4. **Stability** (consistency). Top-k Jaccard and its variance over many seeds and background sets. *Status:* done (Section 4.4, Appendix B).
5. **Robustness.** Change in the explanation under small input perturbations or across models of similar accuracy. *Status:* not done.

Efficiency is measured by runtime (Section 4.2). An explanation should be considered reliable only if it passes several of these checks together; passing a single one is not enough.

5 Conclusion

Our answer to the guiding question is no: a highly accurate prediction does not necessarily come with a reliable explanation. Although the two models reach 98.2% and 95.6% test accuracy, LIME and SHAP share only two of their five most important features on the misclassified instance (Overlap@5 = 0.4). Each method's top-5 also changes between runs, with a mean Jaccard similarity as low as 0.53 for SHAP and 0.72 for LIME. In the deletion test, both methods find influential features far better than a random order, but neither is shown to be more faithful, and neither reveals causal relationships.

Conditions of use. An explanation describes the model's behaviour, either around one instance (LIME) or relative to a baseline (SHAP). Users should report the seed, background data and kernel width, repeat the explanation and report its stability, and apply the checks of Section 4.7, above all for borderline and misclassified instances.

Limitations and future work. With one dataset, three instances, two models and no confidence intervals, the conclusions are illustrative (Appendix C). Next steps are the insertion test, the LIME R^2, TreeSHAP, the log-odds scale, more datasets and more repetitions.

References

[1] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *KDD 2016*, 1135–1144. DOI.

[2] Mersha, M., Lam, K., Wood, J., AlShami, A., & Kalita, J. (2024). Explainable artificial intelligence: A survey of needs, techniques, applications, and future direction. *Neurocomputing*, 599, 128111. DOI (we read the arXiv version 2409.00265v2).

[3] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017*. Paper page.

[4] Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *IS&T/SPIE 1993 International Symposium on Electronic Imaging: Science and Technology*, vol. 1905, 861–870. Paper page.

Data, tools and source of the numbers

- **Data.** Breast Cancer Wisconsin (Diagnostic) [4], loaded with `sklearn.datasets.load_breast_cancer`.
- **Software.** Python with the scikit-learn, `lime` and `shap` libraries.
- **Source of the numbers.** All figures, tables, numbers and both worked examples were produced by the team with the notebook `S04_LIME_SHAP_experiments.ipynb`; none are taken from the cited papers.
- **Use of AI tools.** Claude (Anthropic) was used to help explain concepts, write the experiment code, and draft and revise this report.
- **Verification of AI-assisted content.** We checked every claim about LIME and SHAP against the original papers [1][3] and the survey [2]; re-ran the notebook and compared every number in the tables and text with the CSV files in `results/`; and recomputed both worked examples independently (by hand and with a short NumPy script).
- **Code repository.** The notebook and all results of our experiments are available at `https://github.com/khoiLearnsToCode/CS420_AI_LIME_SHAP`.

A Full experimental setup

B Sensitivity to background size and kernel width

Section 4.4 showed that the explanations change between runs. Here we check whether this instability can be reduced by changing the main parameter of each method: the size of the background set for SHAP and the kernel width for LIME. We use the Random Forest on the borderline instance, the case where LIME is least stable (Jaccard 0.72) and SHAP is also not fully stable (0.73). For each background size we run SHAP on 5 background sets, and for each kernel width we run LIME with 10 seeds. Because LIME uses fewer seeds here than the 20 of Section 4.4, its Jaccard values are not directly comparable with Table 7. Table 9 shows the results.

For SHAP, a background of only 10 instances lowers stability (Jaccard 0.63), but increasing it beyond 50 brings no further gain (0.73). For LIME, a small kernel width of 1.0 is slightly less stable (0.65), and all widths from 2.0 upwards give the same result (0.68). Even the best settings stay well below 1, so tuning these parameters does not remove the instability on this instance.

Table 8: Experimental setup.

Component	Choice	Reason
Data	Breast Cancer Wisconsin [4], loaded from scikit-learn: 569 instances, 30 features, class 0 = malignant, class 1 = benign; 80/20 stratified split with `random_state = 42` (455 train, 114 test)	Public, binary and readily available
Models	Logistic Regression (with standardisation) and a 200-tree Random Forest; test accuracy 98.2% and 95.6%	A linear model as a reference and a less transparent model to expose instability
Explanation space	Probability P(malignant)	LIME explains predicted probabilities [1], so this makes the two methods directly comparable. As a result, the φ values add up on the probability scale, not on the log-odds scale
SHAP	KernelSHAP [3] with 100 background instances drawn from the training set and 2000 sampled coalitions	The same model-agnostic tool for both models, so runtimes are comparable
LIME	5000 perturbed samples, default kernel width $0.75\sqrt{d} \approx 4.11$, default discretisation	The library's standard configuration
Three instances	A fixed rule applied to the RF: the correct prediction with the highest confidence (index 0), the prediction with the smallest $\lvert P - 0.5\rvert$ (index 16) and the first misclassified instance (index 3)	Prevents selecting favourable examples
Number of runs	LIME: 20 seeds; SHAP: 5 background sets (seeds 0–4); top-k with $k = 5$	A compromise between measuring stability and keeping runtime low
Deletion test	20 random test instances (fixed seed); up to 10 features removed by replacing them with the training mean	Checks faithfulness

Table 9: Sensitivity to parameters (Random Forest, borderline instance).

Method	Parameter	Jaccard top-5	Top-1 dominance
SHAP	background = 10	0.63	0.8
SHAP	background = 50	0.73	1.0
SHAP	background = 100 (default)	0.73	1.0
LIME	kernel width = 1.00	0.65	0.7
LIME	kernel width = 2.00	0.68	0.9
LIME	kernel width = 4.11 (default)	0.68	0.9
LIME	kernel width = 8.00	0.68	0.9

C Limitations

- Only one dataset, three instances and two models; the results may not generalise.
- 20 seeds (LIME) and 5 background sets (SHAP) are few, and there are no confidence intervals.
- The deletion test uses one seed and 20 instances, and replacing with the mean can create out-of-distribution samples.
- Runtimes are from one machine and for KernelSHAP; TreeSHAP for Random Forest would be much faster (not measured).

- Explanations are on the probability scale; on log-odds the results may differ (not tested).
- LIME discretises features, so only ranks and signs can be compared with SHAP, not absolute magnitudes.
- There is no ground truth for the Random Forest's explanations, so we cannot say which method is "correct".